

TECHNICAL TEST SUBMISSION

Greenhouse IoT Backend

Reliable sensor ingestion and acknowledged device control through one production-oriented backend.

FastAPI

PostgreSQL

Mosquitto

Docker Compose + Pytest

September 2026

HTTP

sensor input

MQTT

device command

SQL

durable readings

Three behaviors define success

The scope stays deliberately small and reviewable.

01 **POST /sensor-data**

Validate the payload, preserve recorded_at, and persist the reading in PostgreSQL.

02 **POST /device-control**

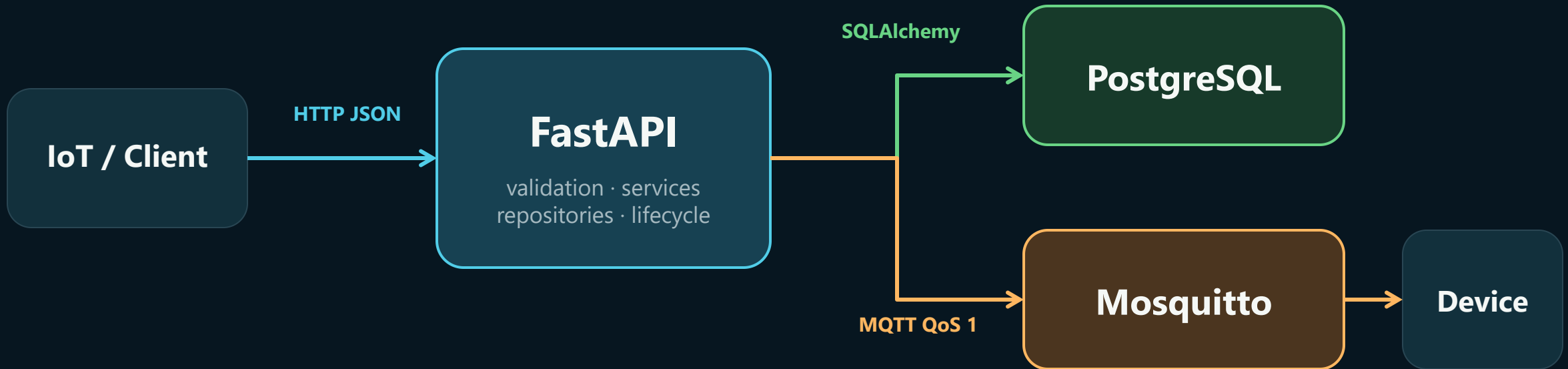
Accept device_id plus ON/OFF and publish a structured command through MQTT QoS 1.

03 **GET /status**

Probe the backend, SELECT 1 against PostgreSQL, and report the live MQTT connection state.

One backend, two transport responsibilities

HTTP handles sensor ingestion. MQTT handles device control.



Requirement interpretation

The implementation contract explicitly makes /sensor-data an HTTP endpoint, while MQTT is reserved for outbound device commands.

The contract is explicit before implementation

OpenAPI reflects the same request, response, and validation rules.

```
...et : #/components/schemas/deviceControlRequest }}, required : true, responses : { 200 : { description : 'Successful response', content : { application/json : { schema :
  { $ref : '#/components/schemas/DeviceControlResponse' } } } }, 422 : { description : 'Unprocessable Entity', content : { application/json : { schema : { $ref : '#/components/schemas/ErrorResponse' } } } }, 503 :
  { description : 'Service Unavailable', content : { application/json : { schema : { $ref : '#/components/schemas/ErrorResponse' } } } } }, { status : { get : { tags : [ 'System' ], summary : 'Get
Status'
{ $ref : '#/components/schemas/DeviceControlRequest' }, { type : 'string', enum :
[ 'ON', 'OFF' ], title : 'Command' }, { type : 'object', required : [ 'device_id', 'command' ], title : 'DeviceControlRequest', description : 'Validated command accepted by POST
/device-control.' }, { type : 'boolean', const : true, title : 'Success', default : true, message : { type : 'string', const : 'Device control command published
successfully', title : 'Message', default : 'Device control command published successfully' }, data : { $ref : '#/components/schemas/PublishedDeviceCommand' }, type : 'object', required :
[ 'data', title : 'DeviceControlResponse', errorBody : { properties : { code : { type : 'string', title : 'Code' }, message : { type : 'string', title : 'Message' }, details : { anyOf : [ { items :
{ $ref : '#/components/schemas/ErrorDetail' }, { type : 'null' } ], title : 'Details' }, type : 'object', required : [ 'code', 'message' ], title : 'ErrorBody', errorDetail : { properties : { field :
{ type : 'string', title : 'Field' }, message : { type : 'string', title : 'Message' }, type : { type : 'string', title : 'Type' }, type : 'object', required :
[ 'field', 'message', 'type' ], title : 'ErrorDetail' }, ErrorResponse : { properties : { success : { type : 'boolean', const : false, title : 'Success', default : false }, error :
{ $ref : '#/components/schemas/ErrorResponse' }, type : 'object', required : [ 'error' ], title : 'ErrorResponse' }, HealthResponse : { properties : { status : { type : 'string', enum :
[ 'healthy', 'degraded' ], title : 'Status' }, services : { $ref : '#/components/schemas/ServiceStatuses' }, type : 'object', required : [ 'status', 'services' ], title : 'HealthResponse', description : 'Overall
health derived from live dependency checks.' }, PublishedDeviceCommand : { properties : { device_id : { type : 'string', title : 'Device Id', command : { type : 'string', enum :
[ 'ON', 'OFF' ], title : 'Command' }, type : 'object', required : [ 'device_id', 'command' ], title : 'PublishedDeviceCommand' }, SensorDataRequest : { properties : { device_id :
{ type : 'string', maxLength : 100, minLength : 1, pattern : '^([A-Za-z0-9]{A-Za-z0-9-}_)*$', title : 'Device Id', temperature : { type : 'number', title : 'Temperature', humidity :
{ type : 'number', maximum : 100.0, minimum : 0.0, title : 'Humidity', timestamp : { type : 'string', format : 'date-time', title : 'Timestamp' }, additionalProperties : false, type : 'object', required :
[ 'device_id', 'temperature', 'humidity', 'timestamp' ], title : 'SensorDataRequest', description : 'Validated payload accepted by POST /sensor-data.' }, SensorDataResponse : { properties : { success :
{ type : 'boolean', const : true, title : 'Success', default : true, message : { type : 'string', const : 'Sensor data stored successfully', title : 'Message', default : 'Sensor data stored
successfully' }, data : { $ref : '#/components/schemas/StoreSensorData' }, type : 'object', required : [ 'data' ], title : 'SensorDataResponse' }, ServiceStatuses : { properties : { backend :
{ type : 'string', const : 'up', title : 'Backend', default : 'up', database : { type : 'string', enum : [ 'connected', 'disconnected' ], title : 'Database' }, mqtt : { type : 'string', enum :
[ 'connected', 'disconnected' ], title : 'MQTT' }, type : 'object', required : [ 'database', 'mqtt' ], title : 'ServiceStatuses', description : 'Status of the backend and each external
dependency.' }, StoreSensorData : { properties : { id : { type : 'integer', exclusiveMinimum : 0.0, title : 'Id' }, type : 'object', required : [ 'id' ], title : 'StoredSensorData' } } } } }
```

Live /openapi.json

POST

/sensor-data

Validate and persist a reading

POST

/device-control

Validate and publish a command

GET

/status

Probe live dependencies

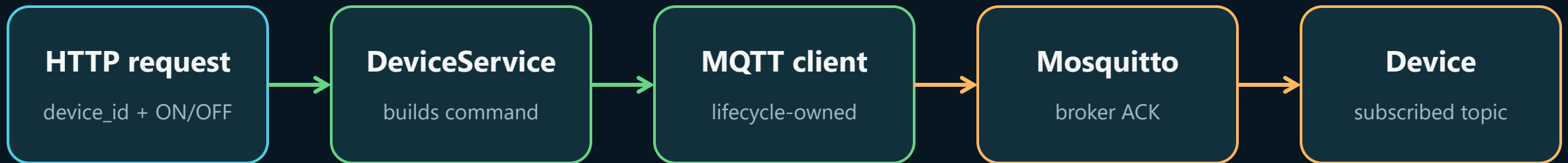
sensor_readings

id	device_id	temperature	humidity	recorded_at	created_at
PK	text	numeric	0–100	sensor time	backend time

Separating recorded_at from created_at preserves network-delay context.

Persistent MQTT keeps commands reliable

The HTTP request returns only after publish acceptance is confirmed.



Topic

greenhouse/control/{device_id}

Payload

```
{ "device_id": "fan-001", "command": "ON",  
  "timestamp": "2026-09-19T10:35:00Z" }
```

Why QoS 1

Broker acknowledgement matters more for control commands than for best-effort telemetry. QoS 1 gives at-least-once delivery, so devices should treat commands idempotently.

Health is derived from real dependency checks

Failures are visible without exposing credentials or stack traces.

HEALTHY

backend up

database connected

mqtt connected

DEGRADED

Any failed DB query or disconnected MQTT client changes the aggregate status.

SELECT 1

+ live client state

Consistent error envelope

```
{
  "success": false,
  "error": {
    "code": "MQTT_UNAVAILABLE",
    "message": "MQTT broker is unavailable"
  }
}
```

Malformed JSON, validation failures, database outages, and broker outages follow the same external structure. Internal detail stays in logs.

Testing proves the critical paths

Automated tests are paired with manual reviewer-style verification.

21 / 21 tests passing

Sensor validation · command validation · MQTT failure · healthy/degraded status

MQTT subscriber evidence

```
$ mosquitto_sub -t greenhouse/control/# -v
```

```
greenhouse/control/fan-001  
{"device_id":"fan-001","command":"ON",...}
```

Database persistence evidence

id	device_id	temperature	humidity	stored through
2	reviewer-sensor	29.4	62.8	POST /sensor-data

Manual integration run retained separate recorded_at and created_at values.

Design choices keep the submission focused

Production expansion is possible without hiding today's failure modes.

Current decisions

FastAPI

Typed validation and automatic OpenAPI with a small code surface.

PostgreSQL + Alembic

Durable relational storage with explicit schema evolution.

Persistent MQTT + QoS 1

Lower request overhead and broker acknowledgement for commands.

Dependency-aware health

Partial failure is surfaced as degraded instead of masked.

Future improvements

Security

TLS, authentication, secret management, and per-device authorization.

Observability

Metrics, tracing, structured log aggregation, and alert thresholds.

Scale

Connection tuning, idempotency keys, and horizontal backend replicas.

Telemetry evolution

Add inbound MQTT consumption only when product requirements demand it.

Submission outcome: one command starts the stack, the API contract stays stable, and failures remain observable.